

UNIVERSIDAD NACIONAL DE CÓRDOBA

Facultad de Ciencias Exactas, Físicas y Naturales

Electrónica Digital II

Informe de Laboratorio N°2

Alumnos:

López Bernal, Julián

Manzo, Mateo

Rivero, Bruno Ignacio

Gordillo, Nahuel Agustin

Lamas, Selena

Índice

1. Introducción.....	3
2. Hardware.....	3
2.1. Asignación y Justificación de Pines.....	3
2.2. Diagrama de Conexión.....	4
2.3. Cálculos Eléctricos.....	4
3. Software.....	5
3.1. Descripción del Algoritmo.....	5
3.2. Diagrama de Flujo.....	7
3.3. Análisis Temporal de Rutinas.....	7
3.4. Código Fuente.....	8
4. Pruebas y Resultados.....	11
5. Conclusiones.....	11

1. Introducción

El presente informe detalla la implementación del Trabajo de Laboratorio N° 2, cuyo objetivo principal es la configuración y uso de terminales del microcontrolador PIC16F887 como entradas y salidas digitales. Se implementaron dos funcionamientos principales combinados: un contador binario de 8 bits y una secuencia luminosa mediante desplazamientos.

Para la visualización se utilizó un banco de 8 LEDs, y para la interacción, un pulsador de propósito general configurado con lógica activa en bajo. El proyecto requirió la correcta configuración de los registros PORTx, TRISx y la desactivación de las entradas analógicas a través de ANSEL/ANSELH. Además, debido a la naturaleza electromecánica del pulsador, se diseñó e implementó una subrutina de anti-rebote por software para garantizar transiciones limpias y evitar falsos incrementos.

2. Hardware

2.1. Asignación y Justificación de Pines

	PIN	Configuracion	
Pulsador	RB0	Entrada digital	Pin configurado como entrada. Al estar en el puerto B, requiere limpiar el registro ANSELH para deshabilitar su entrada analógica asociada (AN12) y permitir lecturas digitales correctas.
LEDs	RD0 a RD7	Salidas digitales	Se utiliza el PORTD completo. Al ser un puerto completo de 8 bits consecutivo, permite escribir directamente el valor de la variable del contador (cont_aux) mediante una única instrucción MOVWF PORTD.

Tabla 1: Asignación de Pines Utilizados

2.2. Diagrama de Conexión

El diagrama se realizó en el software Proteus, donde también se realizó el simulación del mismo para verificar la funcionalidad antes de implementarlo en el circuito físico.

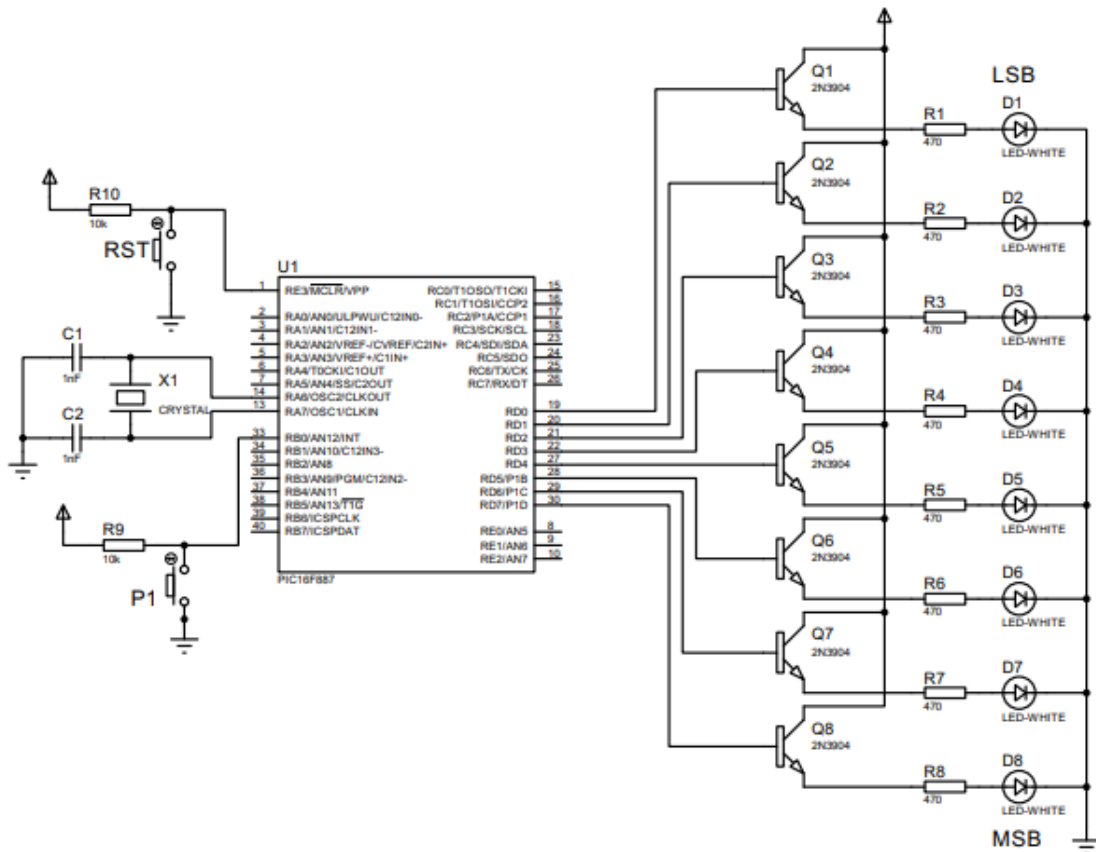


Figura 1: Diagrama de Circuito

2.3. Cálculos Eléctricos

Si bien el consumo de los 8 leds no se ve reflejado en los pines del microcontrolador gracias a la implementación de los transistores NPN 2N3904, igualmente se sobredimensionaron las resistencias limitadoras de los leds ya que la fuente de alimentación utilizada tiene una corriente $I_s = 800 \text{ mA}$ y teniendo en cuenta que el consumo del circuito en versiones futuras va a ir en aumento, se prefiere limitar al máximo la corriente del byte de leds, se tomaron los siguientes datos:

$$V_D = 3 \text{ V}$$

$$I_{MAX} = 40 \text{ mA} \rightarrow I_D = \frac{I_{MAX}}{2} = \frac{40 \text{ mA}}{8} = 5 \text{ mA}$$

Con una corriente de 5 mA (suficiente para que sea visible que el led está encendido) la resistencia limitadora resulta:

$$R_D = \frac{V_S - V_D}{I_D} = \frac{5V - 3V}{0,005A} = 400 \Omega$$

Se utilizó entonces una resistencia para cada diodo de $R_D = 470 \Omega$

$$I_D = \frac{V_S - V_D}{R_D} = \frac{5V - 3V}{470 \Omega} = 4,25 \text{ mA}$$

Para una ganancia del 2N3904 a 5 V de alimentación y 4,25 mA, se tomó el valor aproximado de $h_{FE} = 240$ por lo que la corriente en el puerto D resulta

$$I_B = \frac{I_D}{h_{FE}} = \frac{4,25 \text{ mA}}{240} = 0,0177 \text{ mA} = 17,7 \mu A$$
$$I_{PORTD} = I_B * 8 = 17,7 \mu A * 8 = 141,6 \mu A$$

Por lo que se consume solamente 0,14 mA en todo el puerto, resultado que permite una mayor comodidad a la hora de aumentar el consumo en el microcontrolador.

La resistencia de pull-up externa utilizada para el botón en el pin RB0 es de 10 k Ω

3. Software

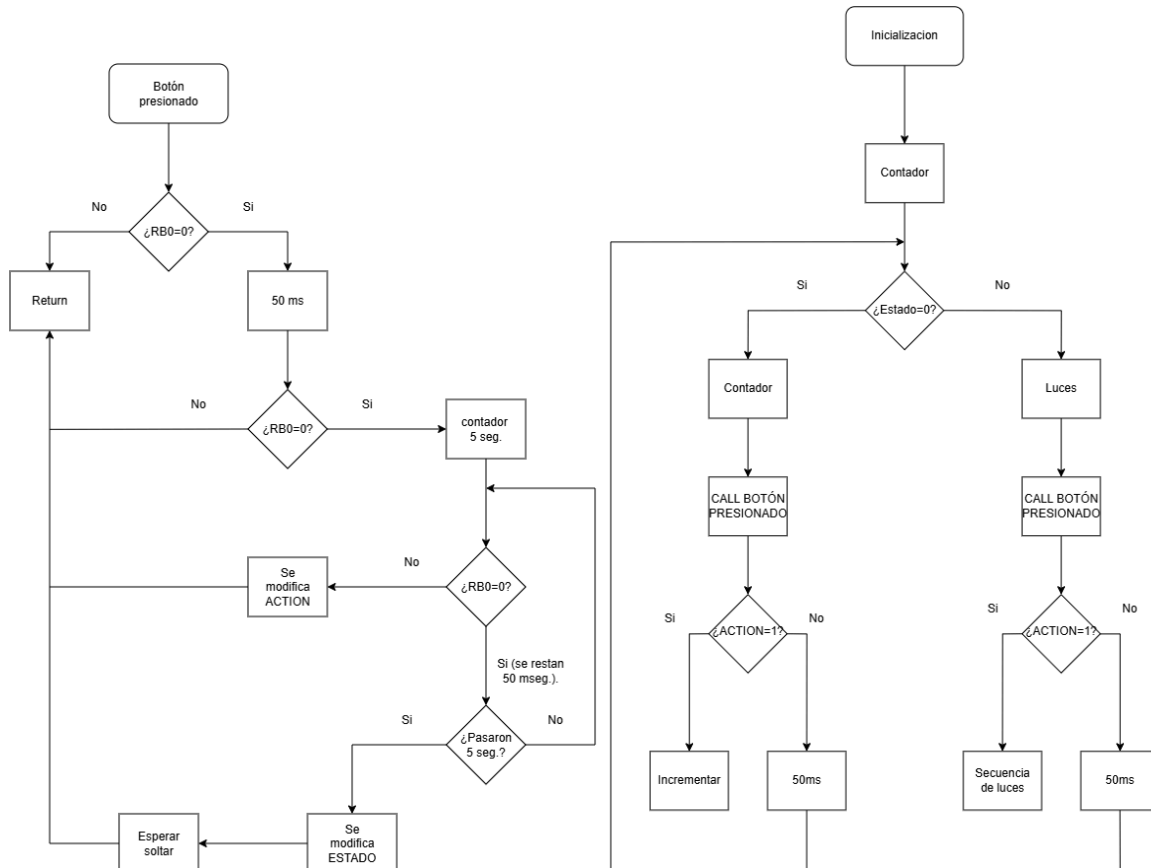
3.1. Descripción del Algoritmo

El programa está estructurado sobre la inicialización de registros de control y un lazo de evaluación continuo.

- Configuración e Inicialización: Al arrancar (ejecutando el salto desde el vector de reset 0x00), el programa pone en modo digital los puertos mediante la limpieza de ANSEL y ANSELH. Configura TRISD como salidas (0x00) y el bit 0 de TRISB como entrada (BSF TRISB, 0). Inicialmente apaga todos los LEDs de PORTD y limpia las variables de memoria RAM var (registro de estados) y cont_aux (registro del contador).
- Registro de Estado (var): La lógica de control se basa en los bits de la variable var ubicada en 0x23:
 - Bit estado (bit 0): Define el modo de funcionamiento. Si está en 0, ejecuta el modo Contador. Si está en 1, ejecuta el modo Luces.
 - Bit action (bit 1): Controla el disparador. En modo Contador, si vale 1 habilita el incremento unitario. En modo Luces, actúa como bandera de habilitación de secuencia (reproducción o pausa).

- MAIN: El programa salta a la sección del modo activo analizando el bit estado de var.
 - En CONTADOR, se llama a la subrutina BOTONPRESIONADO para escanear el puerto. Luego, si el bit action es igual a 1, se ejecuta la subrutina incrementar, posteriormente, pone en 0 el bit action, en cambio, si el bit action está en 0 realiza un delay de 50 ms llamando a la subrutina DELAY50MS
 - INCREMENTAR, aumenta la variable cont_aux en una unidad y actualiza el valor de salida en el PORTD.
 - En LUCES, primeramente se enciende el primer led de la secuencia, luego se llama a BOTONPRESIONADO para verificar si el botón se presionó. Si despues de verificar el pin del boton, si el bit de action está en 1 se desplaza el bit hacia la derecha, sinó, se deja el puerto como está, para simular la “pausa” del sistema, luego se realiza un delay de 100 ms llamando a DELAY50MS y vuelve a la verificación del botón
 - BOTONPRESIONADO, si el boton no está presionado retorna, de otro modo llama a un delay antirrebote y realiza un loop 100 veces en el se llama al delay 50 ms y se verifica si el botón se soltó en cada repetición, si lo hizo se pone en 1 el bit de action si está en modo Contador, y se hace el complemento del bit action si es que está en el modo Luces, en cambio, si pasados los 5 segundos no se soltó el pulsador, se invierte el bit de estado y entra a otro bucle a esperar que se suelte, para luego volver al estado que corresponda.

3.2. Diagrama de Flujo



3.3. Análisis Temporal de Rutinas

La subrutina DELAY50MS utiliza dos bucles anidados con las variables de conteo cont1 (en 0x20) y cont2 (en 0x21). Operando con un cristal oscilador de 4MHz, el tiempo de cada ciclo de instrucción es:

$$F_{CICLO} \frac{4}{F_{osc}} = \frac{4}{4MHz} = 1\mu s$$

Sumando los pocos ciclos de entrada y el RETURN final, la subrutina ejecuta exactamente 5000 CICLOS, lo que equivale a un retardo de 50 ms de extrema precisión lógicamente comprobada mediante la herramienta Stop Watch del software MPLAB

3.4. Código Fuente

```

PROCESSOR 16F887
    ; PIC16F887 Configuration Bit Settings

; Assembly source line config statements

; CONFIG1
    CONFIG FOSC = XT                ; Oscillator Selection bits (XT oscillator: Crystal/resonator on
    RA6/OSC2/CLKOUT and RA7/OSC1/CLKIN)
    CONFIG WDTE = OFF                ; Watchdog Timer Enable bit (WDT disabled and can be enabled by SWDTEN
    bit of the WDTCON register)
    CONFIG PWRTE = OFF                ; Power-up Timer Enable bit (PWRT disabled)
    CONFIG MCLRE = OFF                ; RE3/MCLR pin function select bit (RE3/MCLR pin function is digital
    input, MCLR internally tied to VDD)
    CONFIG CP = OFF                  ; Code Protection bit (Program memory code protection is disabled)
    CONFIG CPD = OFF                  ; Data Code Protection bit (Data memory code protection is disabled)
    CONFIG BOREN = OFF                ; Brown Out Reset Selection bits (BOR disabled)
    CONFIG IESO = OFF                ; Internal External Switchover bit (Internal/External Switchover mode
    is disabled)
    CONFIG FCMEN = OFF                ; Fail-Safe Clock Monitor Enabled bit (Fail-Safe Clock Monitor is
    disabled)
    CONFIG LVP = OFF                  ; Low Voltage Programming Enable bit (RB3 pin has digital I/O, HV on
    MCLR must be used for programming)

; CONFIG2
    CONFIG BOR4V = BOR40V            ; Brown-out Reset Selection bit (Brown-out Reset set to 4.0V)
    CONFIG WRT = OFF                  ; Flash Program Memory Self Write Enable bits (Write protection off)

// config statements should precede project file includes.
#include <xc.inc>

//Variables//
    cont1 EQU 0x20
    cont2 EQU 0x21
    cont5s EQU 0x22
    var EQU 0x23
    estado EQU 1
    action EQU 0
    cont_aux EQU 0x24

PSECT resetVec, class=CODE, delta=2, abs
ORG 0x00
    PAGESEL MAIN
    GOTO MAIN
PSECT code, class=CODE, delta=2
;Acá se va a realizar toda la lógica del programa llamando
;a las subrutinas y entrando en el modo "Luces" o "Contador"
;dependiendo del estado de un bit aux
;bit ESTADO(1) de var en 0 = modo contador
;bit ESTADO(1) de var en 1 = modo luces
;bit ACTION(0) de var en 0 = en luces pausa la secuencia y en contador no hace nada
;bit ACTION(0) de var en 1 = en luces continua la secuencia y en contador incrementa en 1

ORG 0x05
MAIN:
    ;Inicialización de puertos
    BANKSEL ANSEL
    CLRF ANSEL                ; Pone en 0 ANSEL (PORTA y PORTE a modo digital)

```

```

CLRf    ANSELH          ; Pone en 0 ANSELH (PORTB a modo digital)
BANKSEL TRISD
CLRf    TRISD           ; TRISD = 0x00 -> Los 8 pines de PORTD como SALIDAS (LEDs)
BANKSEL TRISB
BSF     TRISB, 0        ; TRISB, bit 0 = 1 -> Pin RB0 como ENTRADA (Pulsador)
BANKSEL PORTD
CLRf    PORTD           ; Apaga todos los LEDs al energizar el circuito
;inicialización de variables
CLRf    var
CLRf    cont_aux

```

CONTADOR:

```

BTFSC var, estado      ;verifica si está en modo Contador
GOTO LUCES             ;si lo está salta esta línea y ejecuta el modo Contador
CALL BOTONPRESIONADO   ;está presionado?
BTFSC var, action       ;si el bit está en 0 no incrementa
CALL INCREMENTAR       ;incrementa en 1 el contador
CALL DELAY50MS         ;espera 50ms
GOTO CONTADOR

```

INCREMENTAR:

```

INCF cont_aux, F       ;incrementa en 1 el contador de leds
MOVF cont_aux, W       ;carga la variable en W
MOVWF PORTD           ;aplico los valores del contador en el puerto D
BCF var, action
GOTO CONTADOR

```

LUCES:

```

MOVLW 0x01
MOVWF PORTD           ;Se enciende el primer led de la secuencia de luces

```

DESPLAZAMIENTO:

```

BTFSS var, estado      ;verifica si está en modo Luces
GOTO CONTADOR         ;si lo está saltea esta línea y ejecuta el modo Luces
CALL BOTONPRESIONADO   ;está presionado?

BTFSC var, action       ;si el bit action está en 1
RRF PORTD, F           ;desplaza todo el bit a la derecha
CALL DELAY50MS
CALL DELAY50MS
GOTO DESPLAZAMIENTO    ;retorna a LUCES para no variar el estado de las luces (Pausa)
;retorna a LUCES para no variar el estado de las luces (Pausa)

```

BOTONPRESIONADO: ;|cambia el valor del bit ACTION si se presiona, y si se presiona|

```

;|durante mas de 5 segundos cambia el valor del bit ESTADO |
BTFSC PORTB, 0        ;Si el boton no está presionado retorna
RETURN
CALL DELAY50MS         ;delay antirrebote
MOVLW 0x63             ;para hacer 5 s se multiplica Delay x ,
MOVWF cont5s

```

BUCLE_ESPERA:

```

BTFSC PORTB, 0        ;si ya se soltó el boton va a la subrutina de boton soltado
GOTO BOTON_SOLTADO
CALL DELAY50MS
DECFSZ cont5s, F       ;hasta que llegue a 0, espera un tiempo
GOTO BUCLE_ESPERA

```

```

MOVLW 0x02
XORWF var, F           ;Si contó los 5 segundos y sigue presionado se invierte el bit de ESTADO

```

```
CLRFR PORTD          ;Apago todos los leds
CLRFR cont_aux        ;Contador en 0
```

```
ESPERAR_HASTA_SOLTAR:      ;espera hasta que el boton efectivamente se suelte
    CALL DELAY50MS
    BTFSS PORTB,0
    GOTO ESPERAR_HASTA_SOLTAR
    BTFSS var,estado
    GOTO CONTADOR          ;si el bit estado está en 0 va a contador
    GOTO LUCES             ;si el bit estado está en 1 va a luces
```

```
BOTON_SOLTADO:
    MOVLW 0x01
    BTFSS var,estado      ;si el sistema está en estado contador
    BSF var,action        ;pone en 1 el bit de acción para que el main haga el cambio
    BTFSC var,estado      ;si el sistema está en estado luces
    XORWF var,F           ;invierte el valor del bit action
    CALL DELAY50MS
    RETURN
```

DELAY50MS:

```
    MOVLW 0x63          ; cont1 = 99
    MOVWF cont1
INICIO_D:
    MOVLW 0xA7          ; cont2 = 167
    MOVWF cont2
LOOP:
    DECFSZ cont2,F       ; Cuento desde 167 a 0
    GOTO LOOP
    DECFSZ cont1,F       ; disminuyo en 1 cont1 y vuelvo a LOOP
    GOTO INICIO_D
    RETURN
```

4. Pruebas y Resultados

Prueba	Lo que se espera	Resultado
Iniciación	Que al alimentar el sistema todos los LEDs en PORTD inicien apagados (PORTD = 0x00) y las variables limpias.	Correcto. Se verificó que las instrucciones CLRFR var y CLRFR cont_aux limpian las variables del mapa de RAM antes de ingresar a MAIN.
Se presiona de forma corta (entrar a contador)	Que al presionar el pulsador en RB0, se active el bit action (bit 1 de var), incrementando la cuenta en cont_aux y actualizando el PORTD.	Se observó el correcto llamado a INCREMENTAR al detectar la pulsación de entrada. El puerto D representa el valor binario exacto en los LEDs.
Se presiona de forma larga (entra a jugo de luces)	Que al mantener presionado el pulsador, el contador interno de 5 segundos (cont5s) llegue a cero y conmute el bit estado (bit 0 de var) para cambiar de modo.	Al mantener el pulsador en bajo por más de 5 segundos, el flujo del programa sale con éxito de CONTADOR y se redirige a LUCES.
Bucle	Que el tiempo entre actualizaciones sea adecuado para evitar rebotes e incrementos múltiples accidentales.	El uso de llamadas encadenadas a DELAY50MS proporciona un excelente control de muestreo de 200 ms en el main.

5. Conclusiones

La realización de este laboratorio permitió afianzar el control directo de periféricos mediante programación en bajo nivel. A través de este código se demostró que el uso correcto de máscaras y bits lógicos de estado (banderas) permite gobernar múltiples modos de operación utilizando un número mínimo de recursos de hardware (en este caso, un único pulsador en RB0).

A nivel metodológico, se destaca la precisión del diseño de retardos temporales mediante bucles de decremento anidados. El cálculo matemático exacto de ciclos de instrucción de la rutina DELAY50MS resolvió eficientemente el control de tiempo del sistema. Asimismo, la estructuración del código en módulos claramente definidos (CONTADOR, LUCES, INCREMENTAR, BOTONPRESIONADO) simplifica la depuración de errores.